# Project 11 – Cooperative Warehouse Robots: Problem Formulation

Real-World RL, Unit 5 · G Karthik Yadav | I Maithreyi | M Sriharshith
100523729022 | 100523729024 | 100523729037

Two warehouse robots learn, with PPO and a shared network, to fetch the shelves customers have requested, carry them to a goal station and put them back without blocking each other.

## Environment

RWARE (Robotic Warehouse, `rware-tiny-2ag-v2`, Gymnasium API) is a 10 x 11 grid with 32 shelves in three columns, 2 goal cells on the bottom edge and a request queue of 2 shelves. A loaded robot cannot drive through other shelves, and robots block each other in the narrow aisles. An episode lasts 500 steps; the team score is the number of shelves delivered.

## Formulation

A cooperative decentralised POMDP: every robot acts on its own local observation, and all robots share one policy.

| Element | Definition |
|---|---|
| State | Robot positions, headings and loads; shelf positions; the 2 requested shelves (hidden from the robots in base RWARE) |
| Observation (base) | 71 numbers per robot: own position, heading, load flag, plus a 3 x 3 patch around it (agents, shelves, requested flag) |
| Observation (+ task hints) | 16 extra robot-frame features: stage (seeking, carrying requested, carrying other), next BFS step toward the target, path length, on-target flag, target vectors to nearest requested shelf, goal and empty slot, cell-ahead-blocked |
| Action | Discrete(5) per robot: no-op, forward, turn left, turn right, toggle load |
| Reward | +1 to the robot that delivers a requested shelf to a goal cell (sparse; team score = deliveries) |
| Horizon, discount | 500 steps (time limit, bootstrapped), gamma = 0.99 |

## Reward and shaping

A random policy delivers almost nothing, so I also test potential-based shaping, which keeps the optimal policy unchanged (Ng, Harada and Russell, 1999):

$$r'_t = r_t + \gamma\, \Phi(s_{t+1}) - \Phi(s_t)$$

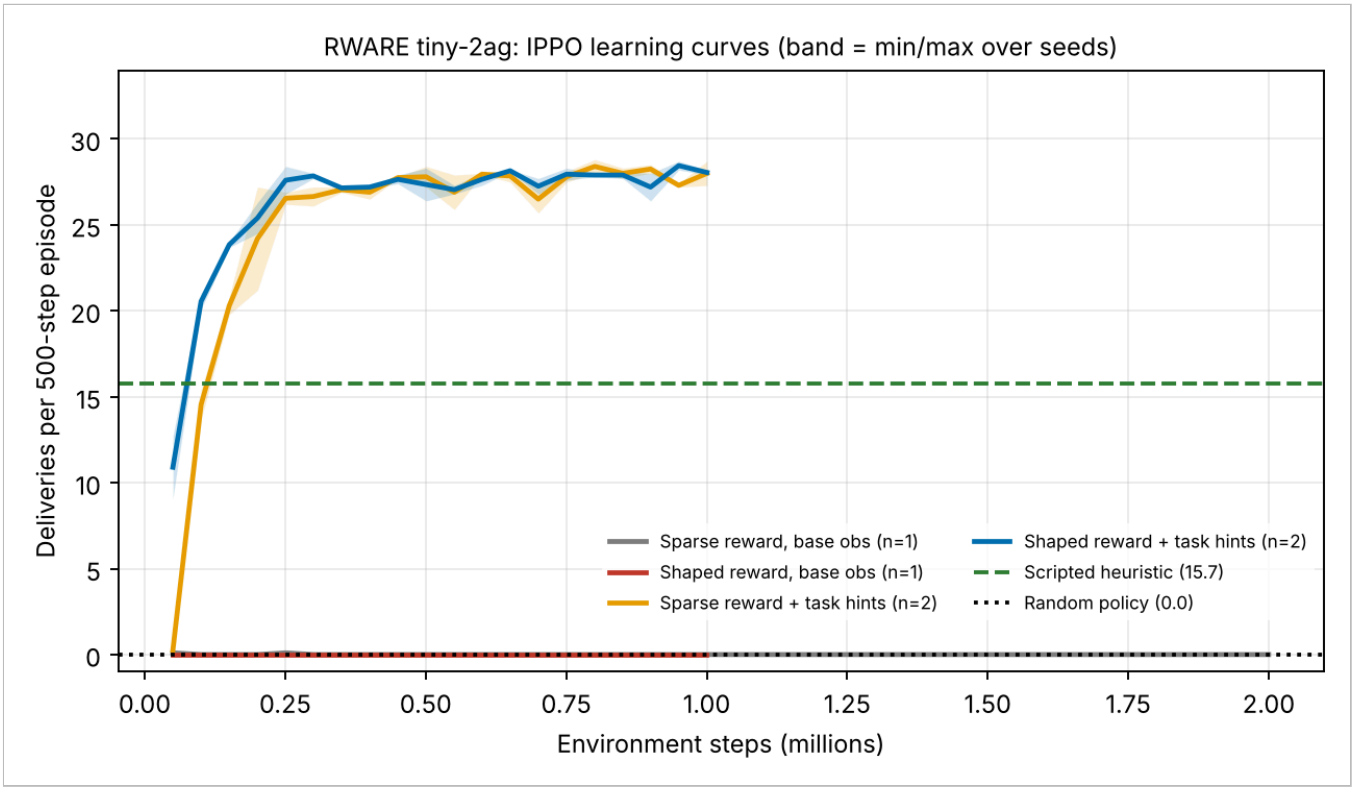
| Stage | Potential Φ (d = BFS path length, D = 21) |
|---|---|
| Seeking a requested shelf | − d(robot, nearest requested shelf) / D |
| Carrying a requested shelf | 1 − d(robot, goal) / D |
| Carrying a non-requested shelf | 0.5 − d(robot, nearest empty slot) / D |

All curves are scored on the unshaped sparse reward.

## Algorithm and baselines

IPPO with parameter sharing: one 2 x 128 tanh actor-critic used by both robots. PPO clip 0.2, GAE λ 0.95, γ 0.99, 4 epochs, minibatch 512, entropy 0.01, Adam 3e-4 decayed linearly; 8 environments x 128 steps per update; 1–1.5M steps per run; time limit bootstrapped from V(final obs). Baselines: random policy and a scripted greedy heuristic with full global state.

# Learning curves




## Results table

| Method | Seeds | Deliveries / episode | Steps |
|---|---|---|---|
| Random policy | - | 0.0 | - |
| Scripted heuristic (global state) | - | 15.7 (episode std 10.0) | - |
| IPPO, sparse reward, base obs | 1 | 0.0 | 2.0M |
| IPPO, shaped reward, base obs | 1 | 0.0 | 1.0M |
| IPPO, sparse reward + task hints | 2 | 26.9 (27.1, 26.8) | 1.0–1.5M |
| IPPO, shaped reward + task hints | 2 | 26.9 (26.4, 27.3) | 1.0–1.5M |

Mean over 50 evaluation episodes per run, sampled actions, sparse reward only. Occasional 0-delivery episodes are deadlocks.

Deliveries per 500-step episode on the sparse-reward evaluation environment. Band = min/max over seeds. IPPO with task hints reaches 24 deliveries within 150k–250k steps and about 27 afterwards; base-observation runs stay at 0 like the random policy; the scripted heuristic scores 15.7.

# Implementation

Core files: environment wrapper, IPPO trainer, baselines.

## `common.py`

```
"""Shared helpers: environment construction, evaluation, random baseline."""
import collections
import gymnasium as gym
import numpy as np
import rware  # noqa: F401  (registers the rware-* environments)
from rware.warehouse import RewardType, _LAYER_SHELFS, _LAYER_AGENTS

ENV_ID = "rware-tiny-2ag-v2"  # 2 robots, 1x3 shelf block, request queue = 2 shelves, 500 steps


def make_env(reward="sparse"):
    """reward='sparse'  -> +1 to the robot that delivers a requested shelf (RWARE default)
       reward='staged'  -> +0.5 on delivery at the goal, +0.5 more when the shelf is returned"""
    rt = RewardType.INDIVIDUAL if reward == "sparse" else RewardType.TWO_STAGE
    return gym.make(ENV_ID, reward_type=rt)


N_AUG = 16
STEP = {0: (0, -1), 3: (1, 0), 1: (0, 1), 2: (-1, 0)}   # Direction value -> (dx, dy)
CW = [0, 3, 1, 2]                                         # UP, RIGHT, DOWN, LEFT (clockwise)


class WarehouseEnv:
    """RWARE wrapper with two independent options (both documented in the report).

    augment=True : append 16 robot-frame 'task-assignment' features (what a warehouse management
                   system would broadcast to each robot).  Base RWARE shows only a 3x3 patch and
                   never reveals which shelves are requested:
        stage one-hot (seeking / carrying requested / carrying other)           3
        next BFS step toward the stage target, in the robot frame (fwd,right,back,left)  4
        standing on the stage target, BFS path length / D                        2
        target vectors (forward, right)/D to nearest requested shelf, goal, empty slot  6
        cell ahead blocked                                                       1
    shaping=True : potential-based shaping F = gamma*Phi(s') - Phi(s)  (Ng et al., 1999),
                   which leaves the optimal policy unchanged:
        stage 0  seeking             Phi = -d(robot, nearest requested shelf)/D
        stage 1  carrying requested  Phi = 1.0 - d_BFS(robot, goal)/D   (shelves block a loaded robot)
        stage 2  carrying other      Phi = 0.5 - d_BFS(robot, empty slot)/D
    The environment reward itself stays the sparse +1 per delivery."""

    def __init__(self, shaping=False, augment=False, gamma=0.99):
        self.env, self.shaping, self.augment, self.gamma = make_env("sparse"), shaping, augment, gamma
        self.u = self.env.unwrapped
        self.action_space = self.env.action_space
        self.H, self.W = self.u.grid_size
        self.D = float(self.H + self.W)

    def _bfs(self, sources, blocked):
        shelf = self.u.grid[_LAYER_SHELFS] != 0
        dist = np.full((self.H, self.W), 1e3)
        q = collections.deque()
        for x, y in sources:
            dist[y, x] = 0; q.append((x, y))
        while q:
            x, y = q.popleft()
            for dx, dy in STEP.values():
                nx, ny = x + dx, y + dy
                if 0 <= nx < self.W and 0 <= ny < self.H and not (blocked and shelf[ny, nx]) \
                        and dist[ny, nx] > dist[y, x] + 1:
                    dist[ny, nx] = dist[y, x] + 1; q.append((nx, ny))
        return dist

    def _nearest(self, a, cells):
        return min(cells, key=lambda c: abs(a.x - c[0]) + abs(a.y - c[1]))

    def _phi_and_feats(self):
        u = self.u
        req = [(s.x, s.y) for s in u.request_queue]
        carried = {id(a.carrying_shelf) for a in u.agents if a.carrying_shelf is not None}
        occupied = {(s.x, s.y) for s in u.shelfs if id(s) not in carried}
        empty = [c for c in self.home if c not in occupied] or list(self.home)
        goals = [tuple(g) for g in u.goals]
        maps = {}
        phis, feats = [], []
        for a in u.agents:
            carrying = a.carrying_shelf is not None
            has_req = carrying and a.carrying_shelf in u.request_queue
            stage = 1 if has_req else (2 if carrying else 0)
            if stage == 0: key, src, blk = "req", req, False
            elif stage == 1: key, src, blk = "goal", goals, True
            else: key, src, blk = "slot", empty, True
            if key not in maps: maps[key] = self._bfs(src, blk)
            dist = maps[key]
            pos = (a.x, a.y)
            at_target = pos in src
            nb = []
            for k, (dx, dy) in STEP.items():
```

```
                nx, ny = a.x + dx, a.y + dy
                if 0 <= nx < self.W and 0 <= ny < self.H:
                    nb.append((dist[ny, nx], k))
            best = min(nb) if nb else (1e3, None)
            d_here = 0.0 if at_target else (dist[a.y, a.x] if not blk else best[0] + 1)
            d_here = d_here if d_here < 1e3 else self.D
            base = (0.0, 1.0, 0.5)[stage]
            phis.append(base - d_here / self.D)
            # ---- features
            hi = CW.index(a.dir.value)
            nxt = np.zeros(4, np.float32)
            if not at_target and best[1] is not None and best[0] < 1e3:
                nxt[(CW.index(best[1]) - hi) % 4] = 1.0
            hx, hy = STEP[a.dir.value]; rx, ry = STEP[CW[(hi + 1) % 4]]
            vec = []
            for t in (self._nearest(a, req), self._nearest(a, goals), self._nearest(a, empty)):
                dx, dy = t[0] - a.x, t[1] - a.y
                vec += [(dx * hx + dy * hy) / self.D, (dx * rx + dy * ry) / self.D]
            ax, ay = a.x + hx, a.y + hy
            blocked_ahead = (not (0 <= ax < self.W and 0 <= ay < self.H)) or bool(u.grid[_LAYER_AGENTS, ay, ax]) \
                or (carrying and bool(u.grid[_LAYER_SHELFS, ay, ax]))
            one = np.zeros(3, np.float32); one[stage] = 1.0
            feats.append(np.concatenate([one, nxt, [float(at_target), d_here / self.D], vec,
                                         [float(blocked_ahead)]]).astype(np.float32))
        return np.array(phis), np.array(feats, dtype=np.float32)

    def _out(self, obs, feats):
        o = np.asarray(obs, dtype=np.float32)
        return np.concatenate([o, feats], 1) if self.augment else o

    def reset(self, **kw):
        obs, info = self.env.reset(**kw)
        self.home = {(s.x, s.y) for s in self.u.shelfs}
        self._p, feats = self._phi_and_feats()
        return self._out(obs, feats), info

    def step(self, actions):
        obs, r, term, trunc, info = self.env.step(actions)
        p2, feats = self._phi_and_feats()
        r = np.asarray(r, dtype=np.float32)
        if self.shaping:
            r = r + self.gamma * p2 - self._p
        self._p = p2
        return self._out(obs, feats), list(r), term, trunc, info

    def close(self):
        self.env.close()


def make_train_env(reward, augment):
    return WarehouseEnv(shaping=(reward == "shaped"), augment=bool(augment))


def stack(obs):
    return np.asarray(obs, dtype=np.float32)


def run_episode(env, policy_fn, seed):
    """Run one episode on a SPARSE env; return number of deliveries (= total team reward)."""
    obs, _ = env.reset(seed=seed)
    total, done = 0.0, False
    while not done:
        acts = policy_fn(stack(obs))
        obs, r, term, trunc, _ = env.step(list(acts))
        total += float(np.sum(r))
        done = term or trunc
    return total


def random_policy(n_agents=2, n_actions=5, rng=None):
    rng = rng or np.random.default_rng(0)
    return lambda obs: rng.integers(0, n_actions, size=n_agents)


def evaluate(policy_fn, n_episodes=20, seed0=10_000, augment=False):
    env = WarehouseEnv(shaping=False, augment=augment)
    scores = [run_episode(env, policy_fn, seed0 + i) for i in range(n_episodes)]
    env.close()
    return float(np.mean(scores)), float(np.std(scores)), scores
```

# train_ippo.py

```
"""Independent PPO with parameter sharing (IPPO / CTDE-free) for RWARE.

All robots share one actor-critic network (each robot feeds its own local observation).
Usage: python train_ippo.py --reward {sparse,shaped} --seed 0 --steps 2000000
"""
import argparse, csv, os, time
import numpy as np
import torch
import torch.nn as nn
from torch.distributions import Categorical

from common import make_train_env, stack, evaluate

p = argparse.ArgumentParser()
p.add_argument("--reward", default="shaped", choices=["sparse", "shaped"])
p.add_argument("--aug", type=int, default=1)   # task-assignment observation features
p.add_argument("--seed", type=int, default=0)
p.add_argument("--steps", type=int, default=2_000_000)   # environment (joint) steps
p.add_argument("--n_envs", type=int, default=8)
p.add_argument("--rollout", type=int, default=128)
p.add_argument("--lr", type=float, default=3e-4)
p.add_argument("--gamma", type=float, default=0.99)
p.add_argument("--lam", type=float, default=0.95)
p.add_argument("--clip", type=float, default=0.2)
p.add_argument("--ent", type=float, default=0.01)
p.add_argument("--epochs", type=int, default=4)
p.add_argument("--mb", type=int, default=512)
p.add_argument("--eval_every", type=int, default=50_000)
p.add_argument("--eval_eps", type=int, default=10)
p.add_argument("--out", default="results")
args = p.parse_args()

torch.set_num_threads(1)
torch.manual_seed(args.seed); np.random.seed(args.seed)
os.makedirs(args.out, exist_ok=True)
tag = f"ippo_{args.reward}_aug{args.aug}_s{args.seed}"


class AC(nn.Module):
    def __init__(s, obs_dim, n_act, h=128):
        super().__init__()
        s.body = nn.Sequential(nn.Linear(obs_dim, h), nn.Tanh(), nn.Linear(h, h), nn.Tanh())
        s.pi, s.v = nn.Linear(h, n_act), nn.Linear(h, 1)

    def forward(s, x):
        z = s.body(x)
        return s.pi(z), s.v(z).squeeze(-1)


envs = [make_train_env(args.reward, args.aug) for _ in range(args.n_envs)]
obs = np.stack([stack(e.reset(seed=args.seed * 1000 + i)[0]) for i, e in enumerate(envs)])  # (E, A, D)
E, A, D = obs.shape
n_act = envs[0].action_space[0].n
net = AC(D, n_act)
opt = torch.optim.Adam(net.parameters(), lr=args.lr, eps=1e-5)


def policy_fn(greedy=False):
    def f(o):
        with torch.no_grad():
            logits, _ = net(torch.as_tensor(o))
            d = Categorical(logits=logits)
            return (logits.argmax(-1) if greedy else d.sample()).numpy()
    return f


T = args.rollout
n_updates = args.steps // (T * E)
log = []
ep_ret, ep_ret_hist = np.zeros(E), []
t0, steps = time.time(), 0

for upd in range(n_updates):
    for g in opt.param_groups:                       # linear LR decay
        g["lr"] = args.lr * (1 - upd / n_updates)
    B_obs = np.zeros((T, E, A, D), np.float32); B_act = np.zeros((T, E, A), np.int64)
    B_lp = np.zeros((T, E, A), np.float32); B_val = np.zeros((T, E, A), np.float32)
    B_rew = np.zeros((T, E, A), np.float32); B_done = np.zeros((T, E), np.float32)
    B_boot = np.zeros((T, E, A), np.float32)  # gamma*V(s_final) added to the reward at time-limit truncation
    for t in range(T):
        with torch.no_grad():
            logits, val = net(torch.as_tensor(obs.reshape(E * A, D)))
            d = Categorical(logits=logits); a = d.sample(); lp = d.log_prob(a)
        a_np = a.numpy().reshape(E, A)
        B_obs[t], B_act[t] = obs, a_np
        B_lp[t], B_val[t] = lp.numpy().reshape(E, A), val.numpy().reshape(E, A)
        for i, e in enumerate(envs):
            o, r, term, trunc, _ = e.step(list(a_np[i]))
            B_rew[t, i] = r; ep_ret[i] += np.sum(r)
            done = term or trunc
            B_done[t, i] = done
            if done:                                 # 500-step time limit = truncation, not a real terminal
                with torch.no_grad():
                    B_boot[t, i] = args.gamma * net(torch.as_tensor(stack(o)))[1].numpy()
```

```
                ep_ret_hist.append(ep_ret[i]); ep_ret[i] = 0
                o, _ = e.reset()
            obs[i] = stack(o)
        steps += E
    with torch.no_grad():
        _, last_val = net(torch.as_tensor(obs.reshape(E * A, D)))
    last_val = last_val.numpy().reshape(E, A)

    # GAE (per agent, episode boundaries shared across the team)
    adv = np.zeros_like(B_rew); lastgae = 0
    for t in reversed(range(T)):
        nv = last_val if t == T - 1 else B_val[t + 1]
        nd = B_done[t][:, None]
        delta = B_rew[t] + B_boot[t] + args.gamma * nv * (1 - nd) - B_val[t]
        lastgae = delta + args.gamma * args.lam * (1 - nd) * lastgae
        adv[t] = lastgae
    ret = adv + B_val

    f_obs = torch.as_tensor(B_obs.reshape(-1, D)); f_act = torch.as_tensor(B_act.reshape(-1))
    f_lp = torch.as_tensor(B_lp.reshape(-1)); f_adv = torch.as_tensor(adv.reshape(-1))
    f_ret = torch.as_tensor(ret.reshape(-1))
    N = f_obs.shape[0]
    for _ in range(args.epochs):
        perm = torch.randperm(N)
        for s in range(0, N, args.mb):
            idx = perm[s:s + args.mb]
            logits, v = net(f_obs[idx]); d = Categorical(logits=logits)
            ratio = (d.log_prob(f_act[idx]) - f_lp[idx]).exp()
            ad = f_adv[idx]; ad = (ad - ad.mean()) / (ad.std() + 1e-8)
            pg = -torch.min(ratio * ad, ratio.clamp(1 - args.clip, 1 + args.clip) * ad).mean()
            vl = 0.5 * (v - f_ret[idx]).pow(2).mean()
            loss = pg + 0.5 * vl - args.ent * d.entropy().mean()
            opt.zero_grad(); loss.backward()
            nn.utils.clip_grad_norm_(net.parameters(), 0.5); opt.step()

    if steps % args.eval_every < T * E or upd == n_updates - 1:
        m, sd, _ = evaluate(policy_fn(), args.eval_eps, seed0=50_000, augment=bool(args.aug))
        tr = float(np.mean(ep_ret_hist[-E:])) if ep_ret_hist else 0.0
        log.append((steps, m, sd, tr))
        print(f"{tag} steps={steps} eval_deliveries={m:.2f}±{sd:.2f} train_ret={tr:.2f} "
              f"{steps / (time.time() - t0):.0f} sps", flush=True)
        with open(f"{args.out}/{tag}.csv", "w", newline="") as f:
            w = csv.writer(f); w.writerow(["steps", "eval_mean", "eval_std", "train_return"]); w.writerows(log)

torch.save(net.state_dict(), f"{args.out}/{tag}.pt")
m, sd, scores = evaluate(policy_fn(), 50, seed0=90_000, augment=bool(args.aug))
mg, sdg, scores_g = evaluate(policy_fn(greedy=True), 50, seed0=90_000, augment=bool(args.aug))
np.savez(f"{args.out}/{tag}_final.npz", stoch=scores, greedy=scores_g)
print(f"FINAL {tag}: stochastic {m:.2f}±{sd:.2f} | greedy {mg:.2f}±{sdg:.2f}", flush=True)
```

# baseline_heuristic.py

```
"""Scripted greedy heuristic (uses privileged global state; a strong reference, not a learner).

Each robot: (1) walk to the nearest *unclaimed* requested shelf and pick it up,
(2) BFS-route to the nearest goal (loaded robots cannot cross shelves),
(3) after delivery, carry the shelf back to the nearest empty slot and drop it, repeat."""
import collections, numpy as np
from rware.warehouse import Direction, _LAYER_SHELFS, _LAYER_AGENTS
from common import make_env, stack, run_episode

NOOP, FWD, LEFT, RIGHT, TOGGLE = range(5)
CW = [Direction.UP, Direction.RIGHT, Direction.DOWN, Direction.LEFT]
STEP = {Direction.UP: (0, -1), Direction.DOWN: (0, 1), Direction.LEFT: (-1, 0), Direction.RIGHT: (1, 0)}


class Heuristic:
    def __init__(self, env):
        self.u = env.unwrapped
        self.home = None

    def _next_cell(self, a, target, loaded):
        u = self.u; H, W = u.grid_size
        shelf = u.grid[_LAYER_SHELFS]; ag = u.grid[_LAYER_AGENTS]
        prev = {(a.x, a.y): None}; q = collections.deque([(a.x, a.y)])
        while q:
            c = q.popleft()
            if c == target: break
            for d in STEP.values():
                n = (c[0] + d[0], c[1] + d[1])
                if not (0 <= n[0] < W and 0 <= n[1] < H) or n in prev: continue
                if loaded and shelf[n[1], n[0]]: continue
                if ag[n[1], n[0]] and n != target: continue          # other robot blocks
                prev[n] = c; q.append(n)
        if target not in prev: return None
        c = target
        while prev[c] != (a.x, a.y) and prev[c] is not None: c = prev[c]
        return c

    def _move(self, a, cell):
        if cell is None: return NOOP
        dx, dy = cell[0] - a.x, cell[1] - a.y
        want = [k for k, v in STEP.items() if v == (dx, dy)][0]
        if a.dir == want: return FWD
        diff = (CW.index(want) - CW.index(a.dir)) % 4
        return RIGHT if diff in (1, 2) else LEFT

    def __call__(self, _obs=None):
        u = self.u
        if self.home is None: self.home = {(s.x, s.y) for s in u.shelfs}
        req = [s for s in u.request_queue]
        carried = {id(a.carrying_shelf) for a in u.agents if a.carrying_shelf is not None}
        occupied = {(s.x, s.y) for s in u.shelfs if id(s) not in carried}
        empty = [c for c in self.home if c not in occupied]
        claimed, acts = set(), []
        for a in u.agents:
            pos = (a.x, a.y)
            if a.carrying_shelf is not None and a.carrying_shelf in req:
                tgt = min(u.goals, key=lambda g: abs(g[0]-a.x)+abs(g[1]-a.y))
                acts.append(NOOP if pos == tgt else self._move(a, self._next_cell(a, tgt, True)))
            elif a.carrying_shelf is not None:
                if pos in empty and pos in self.home: acts.append(TOGGLE); continue
                tgt = min(empty, key=lambda c: abs(c[0]-a.x)+abs(c[1]-a.y)) if empty else pos
                acts.append(self._move(a, self._next_cell(a, tgt, True)))
            else:
                cands = [s for s in req if id(s) not in claimed]
                if not cands: acts.append(NOOP); continue
                s = min(cands, key=lambda s: abs(s.x-a.x)+abs(s.y-a.y)); claimed.add(id(s))
                acts.append(TOGGLE if pos == (s.x, s.y) else self._move(a, self._next_cell(a, (s.x, s.y), False)))
        return acts


if __name__ == "__main__":
    env = make_env("sparse"); h = Heuristic(env)
    scores = []
    for i in range(50):
        h.home = None
        scores.append(run_episode(env, lambda o: h(o), 90_000 + i))
    np.save("results/heuristic_scores.npy", scores)
    print(f"heuristic: {np.mean(scores):.2f} ± {np.std(scores):.2f} deliveries/episode (50 episodes)")
```

# baseline_random.py

```
"""Random-policy baseline on the same 50 evaluation seeds used for the trained policies."""
import numpy as np
from common import evaluate, random_policy
m, sd, scores = evaluate(random_policy(rng=np.random.default_rng(0)), 50, seed0=90_000)
np.save("results/random_scores.npy", scores)
print(f"random baseline: {m:.3f} ± {sd:.3f} deliveries/episode over 50 episodes")
```